.NET FUNDAMENTALS

Personal Details API

Project Structure Guide

.NET 10 | ASP.NET Core Web API | EF Core | SQLite | Swagger / OpenAPI

How the project is organised, what every file does, and how a request flows through the code.

November 2026

Contents

1. The big picture

2. Folder-by-folder tour

3. What happens when the app starts

4. Following one request end to end

5. The data model

6. Validation: three layers

7. Error handling

8. Configuration and environments

9. Database and migrations

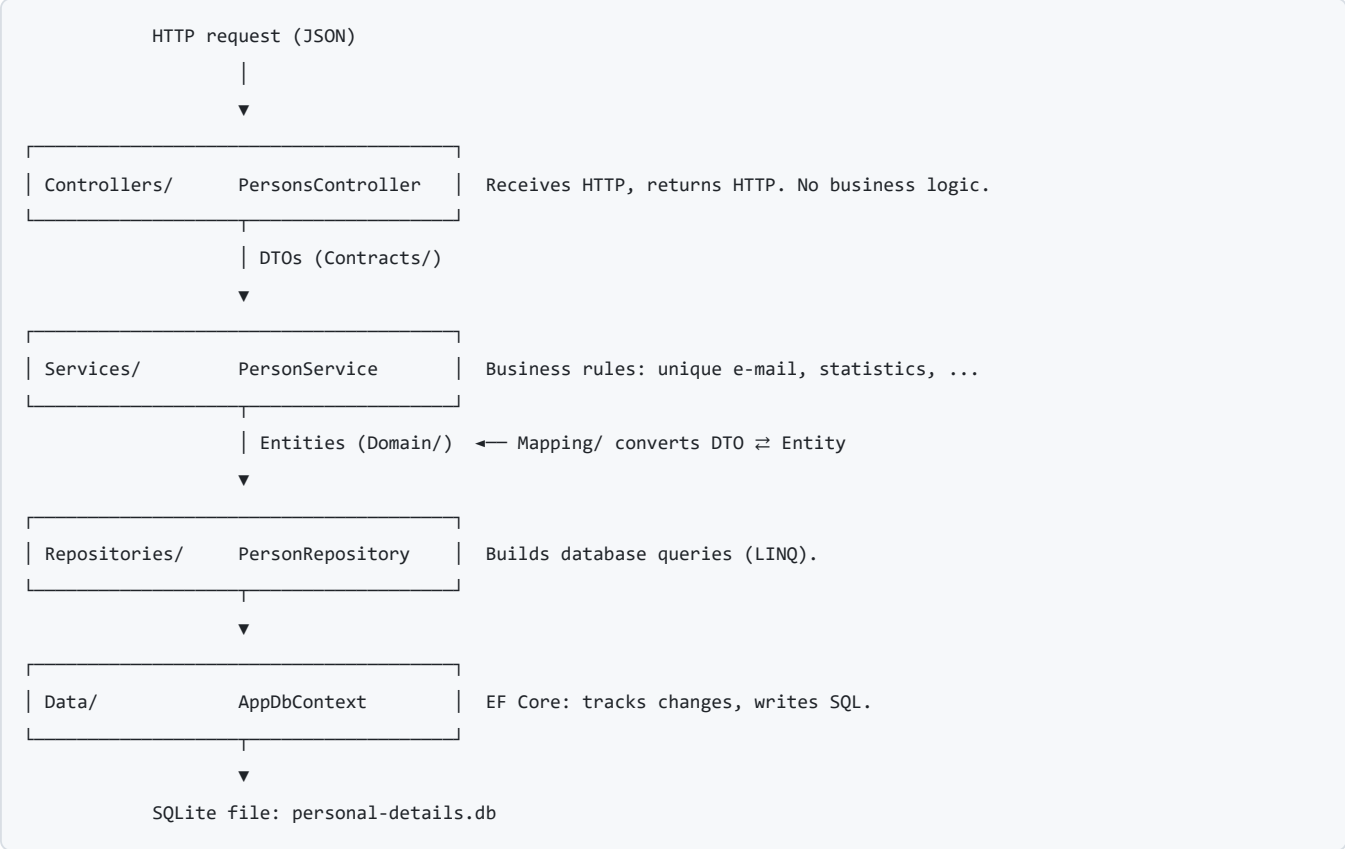
10. How to add a new feature

11. Suggested reading order

12. Glossary

1. The big picture

The project is a **layered** Web API. Each layer has one job and only talks to the layer directly below it.



Code that every layer uses (exceptions, validation attributes, helper extensions) lives in `Common/`. Start-up wiring lives in `Configuration/` and `Program.cs`.

Why split it like this?

Benefit	Example in this project
Each piece is easy to find	Wrong SQL filter? Look in the repository. Wrong HTTP status? Look in the controller or exception.
Layers can change independently	You could swap SQLite for PostgreSQL by changing only `AddPersistence`. Controllers and services don't change.
The API shape is separate from the table shape	`PersonDetailResponse` has `Age` and `MaskedDocumentNumber`, which are not database columns.
Testable	`PersonService` depends on the `IPersonRepository` interface and `TimeProvider`, so a test can pass in fakes.

The two kinds of "model"

Beginners often wonder why there are two similar sets of classes:

- **Entities** (`Domain/`) describe *how data is stored*. EF Core maps them to tables.
- **DTOs** (`Contracts/`), short for *Data Transfer Objects*, describe *what the API accepts and returns*.

Keeping them separate means a client can never set `IsDeleted` or `CreatedAtUtc` directly, and a database change doesn't silently change the public API.

2. Folder-by-folder tour

```
PersonalDetailsApi/
├── Program.cs                      Entry point: registers services, builds the pipeline
├── PersonalDetailsApi.csproj       Project file: target framework, NuGet packages
├── appsettings.json                Settings for all environments
├── appsettings.Development.json    Overrides used when running locally
├── PersonalDetailsApi.http         Sample requests you can send from the editor
├── Properties/launchSettings.json Ports and environment for `dotnet run`
│
├── Controllers/                    HTTP layer
├── Services/                       Business layer
├── Repositories/                   Data-access layer
├── Data/                           EF Core: DbContext, mappings, migrations, seed data
├── Domain/                         Entities and enums (the core model)
├── Contracts/                      Request / response DTOs
├── Mapping/                        DTO ⇄ Entity conversion
├── Common/                         Cross-cutting helpers used by many layers
├── Configuration/                  DI registration and Swagger set-up
└── docs/                           This guide
```

2.1 Root files

File	Purpose
`Program.cs`	The app's entry point. It has three numbered sections: **register services**, **prepare the database**, **configure the HTTP pipeline**. It ends with `public partial class Program;` so future integration tests can reference it.
`PersonalDetailsApi.csproj`	Uses `Microsoft.NET.Sdk.Web`. `GenerateDocumentationFile` turns `///` comments into an XML file that Swagger reads. `TreatWarningsAsErrors` keeps the build clean.
`appsettings.json` / `appsettings.Development.json`	Connection string, seeding switch, Swagger text, log levels. See section 8.
`PersonalDetailsApi.http`	Click "Send Request" in VS Code (REST Client extension) or Visual Studio. The create request is named, so later requests reuse its `id`.
`Properties/launchSettings.json`	Defines the `http` (port 5056) and `https` (port 7284) profiles and sets `ASPNETCORE_ENVIRONMENT=Development`. Only used by `dotnet run` and IDEs, not in production.

2.2 `Controllers/`: the HTTP layer

File	Purpose
`PersonsController.cs`	Six endpoints under `api/v1/persons`. Each method is one line or two: call the service, then wrap the result in `Ok`, `CreatedAtRoute` or `NoContent`.

Things to notice:

- `[ApiController]` turns on automatic model validation. If the body is invalid, the method never runs and a 400 is returned.
- `[Route("api/v1/persons")]` puts the version in the URL.
- `{id:guid}` is a **route constraint**. `/persons/statistics` can't be mistaken for an id because "statistics" isn't a GUID.
- `[ProducesResponseType<T>(...)]` and `///` comments are what Swagger displays. They don't change behaviour.

- `CancellationToken ct` is filled in by ASP.NET Core. If the client disconnects, the database query is cancelled.
- The controller's constructor parameter `IPersonService personService` is a **primary constructor**. DI supplies the service.

2.3 `Services/`: business rules

File	Purpose
`PersonService.cs`	Contains `IPersonService` (the contract) and `PersonService` (the implementation).

Responsibilities:

- **Unique e-mail check** (`EnsureEmailIsUniqueAsync`), which throws `ConflictException` → 409.
- **Not-found handling**: `?? throw new NotFoundException(...)` → 404.
- **Calls mapping** to turn entities into response DTOs.
- **Statistics**: loads a slim projection (`PersonStatRow`) and groups it in memory with LINQ (`GroupBy`, `ToDictionary`). Age groups use a **relational pattern** switch (`< 18 => "Under 18"`).
- **Logging** of create, update and delete with structured placeholders (`{PersonId}`).

The service never touches `HttpContext` and never references EF Core directly. It only knows the repository interface.

2.4 `Repositories/`: data access

File	Purpose
`Repository.cs`	`IRepository<T>` and `Repository<T>`: **generic** CRUD for any `T : BaseEntity`. Written once, reusable for every entity.
`PersonRepository.cs`	`PersonRepository` **inherits** `Repository<Person>` and adds person-specific queries.

Key methods in `PersonRepository`:

- `GetWithDetailsAsync`: `Include`s all five child collections. `AsSplitQuery()` runs one SQL query per collection instead of one huge join. `track: false` adds `AsNoTracking()` for read-only calls, which is faster. Updates pass `track: true` so EF can detect changes.
- `SearchAsync`: builds the query step by step. Each `if` adds a `WHERE`. Nothing runs until `ToPagedResultAsync` (this is called **deferred execution**). Age filters are converted to date-of-birth ranges so SQLite can evaluate them.
- `GetStatRowsAsync`: `Select`s only four columns into a record. This is a **projection**.

2.5 `Data/`: EF Core

File	Purpose
`AppDbContext.cs`	The EF Core context: one `DbSet` per table, plus global rules (see below).
`Configurations/PersonConfiguration.cs`	Fluent API mapping for `Person`: column lengths, indexes, the soft-delete query filter, one-to-many relationships.
`Configurations/PersonDetailsConfiguration.cs`	Mappings for the five child entities. They share a generic base class `PersonOwnedConfiguration<T>` (**template method pattern**). The base adds the query filter; each subclass adds its columns.
`DatabaseInitializer.cs`	Extension method `InitializeDatabaseAsync()`: applies migrations and seeds sample data when enabled. Uses `yield return` to build the sample list.
`Migrations/`	Generated by `dotnet ef`. `*_InitialCreate.cs` holds the `CREATE TABLE` steps. `AppDbContextModelSnapshot.cs` is EF's record of the current model. **Don't edit these by hand.**

Global rules inside `AppDbContext`:

Rule	Where	Effect
Apply all configurations	`OnModelCreating` → `ApplyConfigurationsFromAssembly`	Finds every `IEntityTypeConfiguration<T>` class by reflection.
Ids generated in code	`OnModelCreating` loop → `ValueGeneratedNever()`	Ids are Guid v7 set in `BaseEntity`. Without this rule, a new child added during an update would be treated as an existing row.
Enums as text	`ConfigureConventions`	Columns store `"Female"`, not `1`, so the database is readable.
UTC dates	`ConfigureConventions` → `UtcDateTimeConverter`	SQLite has no time-zone type. This marks values read back as UTC, so the JSON gets its `Z` suffix.
Audit timestamps	`SaveChangesAsync` override	Sets `CreatedAtUtc` / `UpdatedAtUtc` automatically.
Soft delete	`SaveChangesAsync` override	Converts a `Remove()` on an `ISoftDeletable` into an update that sets `IsDeleted = true`.

2.6 `Domain/`: the core model

File	Purpose
`Common/BaseEntity.cs`	`BaseEntity` (abstract; has `Id`), interfaces `IAuditable` and `ISoftDeletable`, and `AuditableEntity`, which combines them.
`Entities/Person.cs`	The **aggregate root**. Holds personal fields and five child lists. `FullName` is computed, not stored.
`Entities/PersonDetails.cs`	`PersonOwnedEntity` (abstract; has `PersonId`) and the five children: `Address`, `EmergencyContact`, `Education`, `Employment`, `IdentityDocument`.
`Enums/Enums.cs`	All enums: `Gender`, `BloodGroup`, `DocumentType`, sort options, and others.

Inheritance tree:

```
BaseEntity (Id)
├── AuditableEntity (+ CreatedAtUtc, UpdatedAtUtc)   implements IAuditable
│       └── Person                                   also implements ISoftDeletable
└── PersonOwnedEntity (+ PersonId, Person)
        ├── Address
        ├── EmergencyContact
        ├── Education
        ├── Employment
        └── IdentityDocument
```

Entities contain **no** references to ASP.NET Core or EF Core. They are plain C# classes.

2.7 `Contracts/`: what the API accepts and returns

File	Purpose
`Requests/PersonRequest.cs`	Body for POST and PUT. Includes nested `AddressRequest`, `EmploymentRequest`, and so on. Contains all validation attributes and cross-field rules. `/// <example>` tags fill in the Swagger examples.
`Requests/PersonQueryParameters.cs`	Query-string options for `GET /persons`: search, filters, sort, page. Bound with `[FromQuery]`.
`Responses/PersonResponses.cs`	Output records: `PersonSummaryResponse` (list rows), `PersonDetailResponse` (full record), child responses, `PersonStatisticsResponse`.

File	Purpose
`Responses/PagedResult.cs`	Generic `PagedResult<T>` with computed `TotalPages`, `HasNextPage`, and a `Map` method to convert the item type.

Requests use `record` with `init` properties so attributes can be placed on them. Responses use **positional records**: one line, immutable, value equality.

2.8 `Mapping/`

File	Purpose
`PersonMappings.cs`	Extension methods: `request.ToEntity()`, `request.ApplyTo(person)`, `person.ToSummary(today)`, `person.ToDetail(today)`.

Mapping is also where clean-up happens: names are trimmed, e-mail is lower-cased, duplicate languages are removed, the first address becomes primary if none is flagged, and document numbers are masked.

The mapping is written by hand rather than with a library like AutoMapper, so every conversion is visible and easy to debug.

2.9 `Common/`: shared helpers

File	Purpose
`Exceptions/AppExceptions.cs`	Abstract `AppException` with `StatusCode` and `Title`. `NotFoundException` (404) and `ConflictException` (409) override them (**polymorphism**).
`Errors/GlobalExceptionHandler.cs`	Implements `IExceptionHandler`. Catches every unhandled exception and writes a `ProblemDetails` JSON response.
`Validation/ValidationAttributes.cs`	Custom attributes `[PastDate]` and `[PhoneNumber]`. The phone check uses a source-generated `[GeneratedRegex]`.
`Extensions/DateExtensions.cs`	`AgeOn(today)`, `TimeProvider.Today()`, `Mask()`, `NullIfWhiteSpace()`.
`Extensions/QueryableExtensions.cs`	Generic `OrderBy(key, direction)`, `ThenBy(key, direction)` and `ToPagedResultAsync(page, size)`. They work for any `IQueryable<T>`.

2.10 `Configuration/`: start-up wiring

File	Purpose
`ServiceCollectionExtensions.cs`	`AddPersistence`, `AddApplicationServices` and `AddApiDocumentation` register services. `UseApiDocumentation` adds Swagger to the pipeline. Keeps `Program.cs` short.
`ApiDocumentationOptions.cs`	Strongly typed settings class bound to the `ApiDocumentation` section (**options pattern**), validated at start-up.

Service lifetimes registered here:

Service	Lifetime	Why
`TimeProvider`	Singleton	Stateless; one instance is enough.
`AppDbContext`	Scoped (set by `AddDbContext`)	One per request, so changes from one request never leak into another.
`IPersonRepository`, `IPersonService`	Scoped	They use the scoped `AppDbContext`, so they can't live longer than it.

3. What happens when the app starts

```
dotnet run
   │
   ▼
Program.cs
 1. WebApplication.CreateBuilder(args)
      reads appsettings.json → appsettings.{Environment}.json → environment variables → command line
 2. builder.Services.AddPersistence(...)        → registers AppDbContext with the SQLite connection string
                    .AddApplicationServices()   → repository, service, TimeProvider, exception handler, health checks
                    .AddApiDocumentation(...)   → options + SwaggerGen
    builder.Services.AddControllers().AddJsonOptions(...)  → enums as strings, omit nulls
 3. builder.Build()                             → DI container is now locked
 4. app.InitializeDatabaseAsync()               → creates a scope, runs migrations, seeds if enabled
 5. Middleware pipeline (runs in this order for every request):
      UseExceptionHandler  → catches errors from everything after it
      UseStatusCodePages   → gives empty 4xx/5xx responses a ProblemDetails body
      UseApiDocumentation  → serves /swagger/v1/swagger.json and the UI at /
      MapControllers       → routes /api/v1/persons/... to PersonsController
      MapHealthChecks      → /health checks that the database is reachable
 6. app.RunAsync()                              → starts Kestrel on http://localhost:5056
```

Order matters: `UseExceptionHandler` comes first so it wraps everything that follows.

4. Following one request end to end

Example: `POST /api/v1/persons` with a JSON body.

Step	Where	What happens
1	Kestrel / middleware	Request arrives and passes through the exception handler and Swagger middleware.
2	Routing	Matches `[HttpPost]` on `PersonsController.Create`.
3	Model binding	The JSON body is deserialised into `PersonRequest`. Enum text like `"Female"` becomes `Gender.Female`.
4	Validation	`[ApiController]` runs all attributes and `IValidatableObject.Validate`. **If anything fails, a 400 `ValidationProblemDetails` is returned here, and steps 5 onwards never run.**
5	Controller	Calls `personService.CreateAsync(request, ct)`.
6	Service	`EnsureEmailIsUniqueAsync` calls `repository.ExistsAsync(...)`. If the e-mail is taken it throws `ConflictException`, which skips to step 11.
7	Mapping	`request.ToEntity()` builds a `Person` with child lists. Each object gets a new Guid v7 id.
8	Repository	`repository.Add(person)` makes EF start tracking the whole graph as *Added*.
9	DbContext	`SaveChangesAsync` sets `CreatedAtUtc` and writes `INSERT` statements for the person and every child in one transaction.
10	Back up	Service maps the entity to `PersonDetailResponse`. The controller returns `CreatedAtRoute`, giving **201** plus a `Location: /api/v1/persons/{id}` header.
11	On error	Any exception bubbles to `GlobalExceptionHandler`, which picks the status code and writes `ProblemDetails`.

Other operations follow the same path:

- **GET by id:** repository loads with `AsNoTracking`, then `ToDetail`.
- **PUT:** loads *tracked* → `ApplyTo` overwrites fields and replaces the child lists → EF works out the `UPDATE`, `DELETE` (removed children) and `INSERT` (new children).
- **DELETE:** `Remove(person)` → `SaveChangesAsync` turns it into `UPDATE Persons SET IsDeleted = 1`.

5. The data model

Six tables. Every child table has a `PersonId` foreign key with cascade delete.



* The e-mail index is **filtered** (`WHERE IsDeleted = 0`), so a soft-deleted person's e-mail can be used again.

Design decisions:

Decision	Reason
Guid v7 ids	Safe to expose in URLs (not guessable like 1, 2, 3) and still time-ordered, so they index well.
Soft delete + global query filter	Data isn't lost by accident. Every query automatically excludes deleted rows. Children of a deleted person are hidden too.
Enums stored as text	The database stays readable with any SQLite viewer.
`Languages` as a JSON column	A simple list of strings doesn't need its own table. EF Core supports primitive collections.
Age not stored	It changes every year. It is calculated on read from `DateOfBirth` and `TimeProvider`.
Computed `IsCurrent`, `FullName`	Derived values are `Ignore`d in the mapping, so they never go out of sync.

6. Validation: three layers

Layer	Where	Examples	Result
1. Attributes on a single field	`PersonRequest` properties	`[Required]`, `[StringLength(100)]`, `[EmailAddress]`, `[Range]`, `[RegularExpression]`, custom `[PastDate]` and `[PhoneNumber]`	400
2. Cross-field rules	`Validate()` in each request record (`IValidatableObject`)	Only one primary address; end date after start date; expiry after issue date; no duplicate documents	400
3. Rules that need the database	`PersonService`	E-mail must be unique among active persons	409

The database is the final safety net: the unique index on `Email` and `NOT NULL` columns protect the data even if code is wrong.

Errors in nested lists come back with their path, for example `"Employments[0].EndDate": ["EndDate cannot be before StartDate."]`.

7. Error handling

All errors use the standard RFC 7807 **ProblemDetails** format:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.5",
  "title": "Resource not found",
  "status": 404,
  "detail": "Person with id '01a1ab38-...' was not found.",
  "instance": "GET /api/v1/persons/01a1ab38-...",
  "traceId": "0HNPR4OMM50MT:00000001"
}
```

How an exception becomes this response:

```
Service: throw new NotFoundException("Person", id)
   │  (bubbles up through the controller; no try/catch anywhere)
   ▼
GlobalExceptionHandler.TryHandleAsync
   switch (exception)
     AppException app          → app.StatusCode, app.Title, app.Message    (404, 409, ...)
     OperationCanceledException → 499 client closed request
     anything else             → 500 "An unexpected error occurred", no detail (never leaks internals)
   logs: Warning for 4xx, Error with stack trace for 5xx
   ▼
IProblemDetailsService writes JSON; AddProblemDetails adds "traceId"
```

To add a new error type, create a class that inherits `AppException` and overrides `StatusCode` and `Title`. You don't need to change the handler.

8. Configuration and environments

ASP.NET Core merges settings in this order, with later sources winning:

```
appsettings.json → appsettings.{Environment}.json → environment variables → command-line args
```

Key	`appsettings.json`	Development override	Meaning
`ConnectionStrings:PersonalDetails`	`Data Source=personal-details.db`	—	SQLite file path
`Database:SeedSampleData`	`false`	`true`	Insert three sample persons if the table is empty
`ApiDocumentation:Enabled`	`false`	`true`	Serve Swagger (off by default for production)
`ApiDocumentation:Title`, `Description`, …	set	—	Text shown in Swagger
`Logging:LogLevel:Microsoft.EntityFrameworkCore.Database.Command`	`Warning`	`Information`	`Information` prints every SQL statement, which is useful for learning

The environment comes from `ASPNETCORE_ENVIRONMENT`, which `launchSettings.json` sets to `Development`.

Examples of overriding settings without editing files:

```
# Use a different database file
dotnet run -- --ConnectionStrings:PersonalDetails="Data Source=other.db"

# Simulate production (Swagger off, no seed data)
$env:ASPNETCORE_ENVIRONMENT = "Production"; dotnet run --launch-profile http
```

`ApiDocumentationOptions` is checked with `ValidateOnStart()`. A missing `Title` stops the app at start-up instead of failing later.

9. Database and migrations

- The database is a single file, `personal-details.db`, in the project folder. It is git-ignored.
- On every start, `MigrateAsync()` applies any migrations that haven't been run yet. EF records applied migrations in the `__EFMigrationsHistory` table.
- **To reset:** stop the app and delete `personal-details.db*`. It is re-created and re-seeded on the next run.
- **To browse it:** use any SQLite viewer, for example the *SQLite Viewer* extension for VS Code or *DB Browser for SQLite*.

Changing the schema:

```
cd PersonalDetailsApi
dotnet tool restore                                          # installs the pinned dotnet-ef
# 1. edit an entity or configuration
dotnet ef migrations add AddSkills --output-dir Data/Migrations # 2. generate the migration
# 3. review the generated Up()/Down() code
dotnet run                                                   # 4. applied automatically at start-up
```

Useful commands:

```
dotnet ef migrations list
dotnet ef migrations has-pending-model-changes   # did I forget to add a migration?
dotnet ef migrations remove                      # undo the last, unapplied migration
dotnet ef migrations script                      # view the SQL
```

10. How to add a new feature

Worked example: add **Skills** (name + proficiency level) to a person. The checklist follows the layers from the bottom up.

#	File	Change
1	`Domain/Enums/Enums.cs`	`public enum Proficiency { Beginner, Intermediate, Advanced, Expert }`
2	`Domain/Entities/PersonDetails.cs`	`public class Skill : PersonOwnedEntity { public string Name { get; set; } = ""; public Proficiency Level { get; set; } }`
3	`Domain/Entities/Person.cs`	`public List<Skill> Skills { get; set; } = [];`
4	`Data/Configurations/PersonDetailsConfiguration.cs`	`public class SkillConfiguration : PersonOwnedConfiguration<Skill>` with `Name` max length. It is picked up automatically.
5	`Data/Configurations/PersonConfiguration.cs`	`builder.HasMany(p => p.Skills).WithOne(s => s.Person).HasForeignKey(s => s.PersonId).OnDelete(DeleteBehavior.Cascade);`
6	`Data/AppDbContext.cs`	`public DbSet<Skill> Skills => Set<Skill>();`
7	Terminal	`dotnet ef migrations add AddSkills --output-dir Data/Migrations`
8	`Contracts/Requests/PersonRequest.cs`	`SkillRequest` record with validation, and `List<SkillRequest> Skills` on `PersonRequest`
9	`Contracts/Responses/PersonResponses.cs`	`SkillResponse` record, and add it to `PersonDetailResponse`
10	`Mapping/PersonMappings.cs`	One `Replace(p.Skills, r.Skills, ...)` line in `ApplyTo`, and the mapping in `ToDetail`
11	`Repositories/PersonRepository.cs`	`.Include(p => p.Skills)` in `GetWithDetailsAsync`

The controller and service need **no changes**, because they work with the whole person. This is the payoff of the layering.

For a **new top-level resource** (for example "Departments"), you'd add an entity, a configuration, a `DbSet`, a migration, DTOs, a mapping, an `I…Repository` (inheriting `IRepository<T>`), an `I…Service`, a controller, and two `AddScoped` lines in `AddApplicationServices`.

11. Suggested reading order

If you are new to the code, read the files in this order. Each one builds on the previous.

1. `Program.cs` : the overall shape
2. `Configuration/ServiceCollectionExtensions.cs` : what gets registered
3. `Domain/Common/BaseEntity.cs` → `Domain/Entities/Person.cs` → `PersonDetails.cs` : the model
4. `Controllers/PersonsController.cs` : the API surface
5. `Contracts/Requests/PersonRequest.cs` : input and validation
6. `Services/PersonService.cs` : business rules
7. `Mapping/PersonMappings.cs` : conversions
8. `Repositories/Repository.cs` → `PersonRepository.cs` : queries
9. `Data/AppDbContext.cs` → `Configurations/` : persistence
10. `Common/Errors/GlobalExceptionHandler.cs` : error handling

Hands-on tip: run the app in Development, then send requests from `PersonalDetailsApi.http` or Swagger and watch the console. Each SQL statement EF generates is logged, so you can see the LINQ in `PersonRepository` turn into SQL.

12. Glossary

Term	Meaning here
Aggregate root	The main entity (`Person`) that owns its children. Children are always loaded and saved through it.
DTO	Data Transfer Object: a class shaped for the API, not the database (`Contracts/`).
Entity	A class EF Core maps to a table (`Domain/Entities/`).
DbContext	EF Core's session with the database. It tracks changes and turns `SaveChanges` into SQL.
Migration	Generated C# code that changes the schema from one version to the next.
Fluent API	Configuring EF with C# method calls (`builder.Property(...).HasMaxLength(100)`) instead of attributes.
Tracking / AsNoTracking	Whether EF remembers loaded entities so it can detect changes. Turn it off for read-only queries.
Global query filter	A `WHERE` clause EF adds to every query for an entity (`!IsDeleted`).
Soft delete	Marking a row as deleted instead of removing it.
Dependency Injection (DI)	The framework creates objects and passes their dependencies through constructors.
Scoped / Singleton	DI lifetimes: one instance per HTTP request, or one for the whole app.
Middleware	A component in the request pipeline (`UseExceptionHandler` , `MapControllers` , ...).
ProblemDetails	Standard JSON error format (RFC 7807).
Options pattern	Binding a configuration section to a typed class (`ApiDocumentationOptions`).
Deferred execution	A LINQ query only runs when its results are enumerated (`ToListAsync` , `CountAsync`).
Projection	Selecting only the needed columns into a new shape (`Select(p => new PersonStatRow(...))`).